PYTHON PROGRAMMING – ASSIGNMENT 1

DESIGN, IMPLEMENT AND VALIDATE A PAYMENT CALCULATOR FOR A VILLAGE MILK CO-OPERATIVE

Ksheera Village Milk Producers' Co-operative

Course Code: CSA08 | CO1 | Total Marks: 50

Student Name: Anshika A

Register Number: 1920260027

Faculty: Dr Haripriya Anandakumar

GitHub Link:

Submission Date:

Important note

This is a Word-ready report draft with illustrative outputs. Execute all programs yourself, add screenshots, replace benchmark placeholders with your actual measurements, and add your working GitHub URL. Do not report illustrative timings as measured evidence.

Introduction

The Ksheera co-operative pays farmers according to litres supplied, fat percentage and solids-not-fat (SNF) percentage. The calculator must validate inputs, calculate the rate consistently, avoid money errors, and support seasonal revisions.

Assumptions and rules

- Base rate ₹30.00/L at 3.5% fat and 8.5% SNF.
- Each 0.1 percentage-point fat step changes the rate by ₹0.40; each SNF step changes it by ₹0.30.
- Fat range: 2.0–7.0%, inclusive. Assume SNF range 6.0–10.0%, inclusive, until the co-operative confirms it.
- Litres must be positive; farmer ID cannot be blank; shift is Morning or Evening.
- Use Decimal created from strings and ROUND_HALF_UP to round each farmer payment to two decimal places. Sum rounded payments for the ten-day total.
- Task 12 changes April–June base rate to ₹32.00, fat step value to ₹0.45, and adds ₹0.50/L if ten-day supply is strictly greater than 500 L. SNF step value remains ₹0.30 by assumption.

TASK 1 – Understand the Problem and Identify the Data (4 marks)

Data	Type	Reason
Farmer ID	str	May include letters or leading zeroes.
Litres	Decimal	Fractional quantity; decimal text is preserved exactly.
Fat and SNF	Decimal	Fractional percentages used in the rate calculation.
Shift	str	Morning or Evening collection.
Step counts	int	Whole number of 0.1 percentage-point steps.
Rate, payment, total	Decimal	Money values must be rounded to paisa.

```
BASE_RATE = Decimal("30.00")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP_SIZE = Decimal("0.1")
```

Separate variables such as litres1, fat1, snf1 through litres300, fat300, snf300 are repetitive and difficult to scale. A list of records lets one loop process any number n of entries. Both approaches take O(n) time; a list uses O(n) space, while separate variables also require O(n) storage but are much harder to maintain.

TASK 2 – Explore Values and Types in the Interpreter (4 marks)

```
>>> type(10)
<class "int">
>>> type(10.0)
<class "float">
>>> type("10")
<class "str">
>>> type(True)
<class "bool">
>>> type([10, 8.5])
<class "list">
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

/ gives true division; // floors the result; % returns the remainder. Negative floor division rounds down, so -7 // 2 is -4. Binary floats cannot exactly represent many decimal fractions.

Complete the Code: Decimal total

```
from decimal import Decimal
payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

```
Expected output:
float  : 33400.30000000061
Decimal: 33400.30
False True
```

Decimal(amount) preserves the original decimal string. Decimal(float(amount)) first imports the float approximation, so the approximation error remains. Integer paise is another exact method: ₹33.40 becomes 3340 paise. float plus repeated round() is less reliable for accounting. All totals take O(n) time and O(1) accumulator space.

```
import sys
from decimal import Decimal
print(sys.getsizeof(3340))
print(sys.getsizeof(33.40))
print(sys.getsizeof(Decimal("33.40")))
```

Run this code and record the memory sizes printed on your own Python version; do not invent the measurements.

TASK 3 – Choose Variables and Name Constants (3 marks)

Use descriptive lowercase_with_underscores names for variables and UPPER_CASE for constants. Python does not enforce constants; uppercase is a convention. Dynamic typing allows x = 10 followed by x = "milk". Swapping is simple: a, b = b, a.

Magic numbers may require k edits when a rate appears in k places. Named constants require one edit. A RATE_CARD dictionary groups rates and supports seasonal selection. For a fixed rate card, runtime remains O(1); the dictionary uses a small O(1) amount of extra space.

TASK 4 – Write Correct Expressions (4 marks)

```
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
payment = litres * rate
```

```
>>> (4.2 - 3.5) / 0.1
7.000000000000001
>>> round((4.2 - 3.5) / 0.1)
7
>>> int(-1.8)
-1
>>> math.floor(-1.8)
-2
>>> round(-1.8)
-2
```

Three surprises: 0.1 + 0.2 == 0.3 is False because of binary representation; the step expression may produce 7.000000000000001; and -7 // 2 is -4 because floor division rounds down. ** is right-associative: 2 ** 3 ** 2 is

512. Unary minus means -2 ** 2 is -4 but (-2) ** 2 is 4. Chained comparison 2.0 <= fat <= 7.0 checks both limits.

For fat 4.2, the mathematical step count is +7; for fat 3.1 it is -4. int() truncates toward zero and floor() rounds down; float error can make the computed positive count 6 or the negative count -3 under int(). round() is preferable to truncation, but Decimal arithmetic is safer. All methods are O(1) time and space.

TASK 5 – Read Input and Convert Types (3 marks)

```
from decimal import Decimal, InvalidOperation

def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print("Invalid number. Example: 8.5")

amount = Decimal("283.90")
print(f"Amount: Rs {amount:8.2f}")
```

input() returns a string. int() and Decimal() convert it. "8,5" and "eight" are rejected as invalid numeric text; " 8.5 " is accepted after strip(). Expected formatting: Amount: Rs 283.90. try/except is clearer than trying to anticipate every invalid string with string methods. Normal validation is O(1) per entry.

TASK 6 – Organise Logic into Functions (4 marks)

```
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE = Decimal("30.00")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP_SIZE = Decimal("0.1")

def rate_per_litre(fat, snf):
    """Return the price per litre rounded to cents."""
    f = int(((fat-STANDARD_FAT)/STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    s = int(((snf-STANDARD_SNF)/STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    return (BASE_RATE + f*FAT_STEP_VALUE + s*SNF_STEP_VALUE).quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Return one rounded payment."""
    return (litres * rate_per_litre(fat, snf)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def ten_day_total(payments):
```

```
    """Sum already-rounded payments."""
    return sum(payments, Decimal("0.00")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```

return passes a value to the caller; print only displays it. Function variables are local; constants may be read but should not be modified inside a function. A long script duplicates logic; lambdas are not suitable for multi-step calculations. Function overhead is small and the overall processing remains O(n).

TASK 7 – Build a Module (4 marks)

Save the following as milkrate.py. The main guard prints the example table only when run directly.

```
"""Milk co-operative rate card module."""
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")

def rate_per_litre(fat, snf):
    """Return rate per litre rounded to cents."""
    f = int(((fat-STANDARD_FAT)/STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    s = int(((snf-STANDARD_SNF)/STEP_SIZE).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    return (BASE_RATE+f*FAT_STEP_VALUE+s*SNF_STEP_VALUE).quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Return the rounded payment."""
    return (litres*rate_per_litre(fat,snf)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

if __name__ == "__main__":
    for l,f,s in
[(Decimal("10"),Decimal("3.5"),Decimal("8.5")),(Decimal("8.5"),Decimal("4.2"),Decimal("8.7")),(Decimal(
"12"),Decimal("3.1"),Decimal("8.3"))]:
        print(f"{l:5} L  fat {f}  snf {s}  rate {rate_per_litre(f,s):6.2f}  pay {payout(l,f,s):8.2f}")
```

```
import milkrate as mr
from decimal import Decimal
print(mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")))
Expected import-test output: 283.90
```

import milkrate uses the module name; from milkrate import payout imports one function; import milkrate as mr uses an alias. Without the main guard, the demonstration table would print on import. Python searches sys.path and caches modules in sys.modules. Copying functions into every script increases update work; a module centralises logic. Import cost occurs once per process; calculation per entry is O(1).

TASK 8 – Plan the Algorithm Before Coding (4 marks)

Pseudocode

```
BEGIN
  total = 0.00
  FOR each entry
    READ farmer ID, litres, fat, SNF, shift
    IF invalid or outside limits THEN show error and re-enter
    ELSE
      fat_steps = nearest_integer((fat - 3.5) / 0.1)
      snf_steps = nearest_integer((SNF - 8.5) / 0.1)
      rate = 30.00 + fat_steps*0.40 + snf_steps*0.30
      payment = round_to_paisa(litres*rate)
      display payment
      total = total + payment
    END IF
  END FOR
  display total
END
```

Flowchart for validating one entry

[Start] -> [Read entry] -> <Valid and in range?>
No -> [Display error / re-enter] -> [Read entry]
Yes -> [Calculate steps and rate] -> [Calculate payment]
-> [Display payment and add to total] -> [End entry]

Entry	Litres	Fat	SNF	Rate	Payment
1	10	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90
3	12	3.1	8.3	27.80	333.60
Total					917.50

Algorithmic problem solving: understand, plan, write, test. A list of entries uses O(n) time and O(n) space. A running total uses O(n) time and O(1) extra space. Store entries when auditing, corrections, or later reports are needed.

TASK 9 – Implement and Test the Calculator (4 marks)

```
from decimal import Decimal
from milkrate import rate_per_litre, payout
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("6.0"), Decimal("10.0")

def valid_entry(fid, litres, fat, snf, shift):
    """Validate one entry against the assignment assumptions."""
    return bool(fid.strip()) and litres > 0 and MIN_FAT <= fat <= MAX_FAT and MIN_SNF <= snf <=
MAX_SNF and shift.lower() in ("morning", "evening")

def calculate_batch(entries):
```

```
    """Return results and the total for valid entries."""
    results, total = [], Decimal("0.00")
    for fid, litres, fat, snf, shift in entries:
        if not valid_entry(fid, litres, fat, snf, shift):
            results.append((fid, "INVALID"))
        else:
            amount = payout(litres, fat, snf)
            results.append((fid, amount)); total += amount
    return results, total.quantize(Decimal("0.01"))

if __name__ == "__main__":
    sample =
[("F001",Decimal("10"),Decimal("3.5"),Decimal("8.5"),"Morning"),("F002",Decimal("8.5"),Decimal("4.2"),
Decimal("8.7"),"Evening")]
    print(calculate_batch(sample))
```

Automated tests

```
from decimal import Decimal
from milkrate import rate_per_litre, payout
assert rate_per_litre(Decimal("3.5"),Decimal("8.5")) == Decimal("30.00")
assert rate_per_litre(Decimal("4.2"),Decimal("8.7")) == Decimal("33.40")
assert rate_per_litre(Decimal("3.1"),Decimal("8.3")) == Decimal("27.80")
assert payout(Decimal("10"),Decimal("3.5"),Decimal("8.5")) == Decimal("300.00")
assert payout(Decimal("8.5"),Decimal("4.2"),Decimal("8.7")) == Decimal("283.90")
print("All assertions passed")
```

Expected output: All assertions passed

Test	Expected	Actual
Standard composition	₹300.00	Fill after running
Above standard	₹283.90	Fill after running
Below standard	₹333.60	Fill after running
Fat boundary 2.0%	Accepted	Fill after running
Fat boundary 7.0%	Accepted	Fill after running
Negative litres	Rejected	Fill after running
Fat 7.1%	Rejected	Fill after running
Text "eight"	Rejected during conversion	Fill after running
Blank ID	Rejected	Fill after running

Manual testing is easy to start but repeated typing is slow and inconsistent. assert tests can be rerun after each change. For m tests, a run is O(m). Use manual tests for interactive prompts and formatting as well.

TASK 10 – Compare Expectations with Execution (4 marks)

```
from decimal import Decimal
from time import perf_counter

def measure(n):
    values = ["33.40"] * n
    start = perf_counter(); ftotal = sum(float(x) for x in values); ft = perf_counter()-start
```

```
    start = perf_counter(); dtotal = sum((Decimal(x) for x in values), Decimal("0.00")); dt = perf_counter()-
start
    print("n=", n, "float=", ftotal, "Decimal=", dtotal, "float_seconds=", ft, "decimal_seconds=", dt)

for n in (300, 6000, 150000): measure(n)
```

Run and record your actual output for 300, 6,000, and 150,000 payments. Timing depends on hardware and background load. The exact Decimal total for repeated ₹33.40 is ₹33.40 × n. Float totals may show a small discrepancy. Rounding each farmer payment before summing matches a per-farmer bank payment; rounding only the grand total may differ.

A dictionary rate cache uses expected O(1) lookup for each of k distinct fat/SNF pairs and O(k) extra space. Recalculation is also O(1) per entry, so caching may not help unless pairs repeat frequently. Benchmark both with time.perf_counter() or timeit and report actual results.

TASK 11 – Recommend and Defend the Final Design (4 marks)

Recommend Decimal created from input strings, ROUND_HALF_UP to two decimal places for each payment, a central rate module, validated inputs, functions, and assert-based tests. A rate revision should change one rate card or validated configuration file. The design processes n entries in O(n) time; keeping all records requires O(n) space, while a running total needs O(1) additional space.

Limitations: this command-line program is not a database-backed production system and does not yet address simultaneous clerks, duplicate records, audit trails, access control, or backups. At 150,000 entries, O(n) calculation is reasonable in principle, but production deployment requires database and audit safeguards.

If rates are read from JSON, loading t values takes O(t) startup time and O(t) memory. It makes revisions easier without editing source code, but the file must be validated and changes audited. Hard-coded constants are simpler but require program edits and redeployment.

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

A. Impact analysis (2 marks)

The analyser now reports two decimal places. April–June uses base ₹32.00 and fat step ₹0.45. If ten-day supply is greater than 500 L, add ₹0.50 for every litre supplied in that ten-day period. The functions must accept a season and calculate the bonus from the ten-day quantity. Two-decimal readings may fall between 0.1% steps; define nearest-step rounding clearly. The SNF step remains ₹0.30 as an explicit assumption.

B. Evaluate the original design (2 marks)

Modify rather than retain unchanged or completely redesign. The original modular structure, Decimal arithmetic, validation and tests remain useful, but fixed constants must become selectable rate cards. Pass the season into the rate function. Keep the bonus in a separate function.

C. Revised implementation (2 marks)

```
from decimal import Decimal, ROUND_HALF_UP
STEP=Decimal("0.1"); CENT=Decimal("0.01")
STANDARD_FAT=Decimal("3.5"); STANDARD_SNF=Decimal("8.5")
SNF_STEP=Decimal("0.30"); BONUS_RATE=Decimal("0.50")
RATE_CARDS={"regular":{"base":Decimal("30.00"),"fat":Decimal("0.40")},
        "apr_jun":{"base":Decimal("32.00"),"fat":Decimal("0.45")}}

def nearest_step(x):
    """Round Decimal step count to nearest integer, ties away from zero."""
    return int(x.quantize(Decimal("1"),rounding=ROUND_HALF_UP))

def revised_rate(fat,snf,season):
    """Calculate rate for the selected seasonal card."""
    card=RATE_CARDS[season]
    fs=nearest_step((fat-STANDARD_FAT)/STEP)
    ss=nearest_step((snf-STANDARD_SNF)/STEP)
    return
(card["base"]+fs*card["fat"]+ss*SNF_STEP).quantize(CENT,rounding=ROUND_HALF_UP)

def revised_payment(litres,fat,snf,season):
    """Calculate one payment before bonus."""
    return (litres*revised_rate(fat,snf,season)).quantize(CENT,rounding=ROUND_HALF_UP)

def bonus_for_ten_days(total_litres):
    """Return bonus only when ten-day litres exceed 500."""
    if total_litres > Decimal("500"):
        return (total_litres*BONUS_RATE).quantize(CENT,rounding=ROUND_HALF_UP)
    return Decimal("0.00")

if __name__ == "__main__":
    for season in ("regular","apr_jun"):
        print(season,revised_rate(Decimal("4.20"),Decimal("8.70"),season))
    print("bonus 500 L:",bonus_for_ten_days(Decimal("500")))
    print("bonus 550 L:",bonus_for_ten_days(Decimal("550")))
```

Illustrative expected output:
regular 33.40
apr_jun 35.75
bonus 500 L: 0.00
bonus 550 L: 275.00

D. Evidence-based defence (2 marks)

Test	Expected	Purpose
Regular season, fat 4.20, SNF 8.70	₹33.40/L	Baseline card
April–June, same readings	₹35.75/L	Seasonal base and fat step
Ten-day total 500 L	No bonus	Strictly greater-than condition
Ten-day total 500.01 L	Bonus ₹250.01	Boundary just above threshold
Ten-day total 550 L	Bonus ₹275.00	Bonus calculation

Run these cases and add screenshots. Decision: MODIFY. This reuses the original reliable structure while adding a seasonal rate card and a separate bonus function. The trade-off is slightly more code and a need to test each rate card and threshold.

GitHub repository and README checklist

CSA08-<RegisterNumber>-Assignment1/
README.md
requirements.txt
Task01/ ... Task12/
Task07/milkrate.py
Task09/test_calculator.py
Task10/benchmark.py
Task12/revised_calculator.py

Ksheera Milk Payment Calculator
Student name: [your name]
Register number: [your number]
Python version: [run python --version]
Libraries: Python standard library (decimal, time, sys)

Run each task using python filename.py.
Add a short result note for every task and actual benchmark measurements.

Keep one folder per task with source files, inputs and actual output screenshots. Commit progressively, at least one commit per task. Verify faculty access to the repository before submission.

Conclusion

The calculator combines appropriate Python types, named constants, expressions, functions, modules, validation, and algorithmic problem solving. Decimal arithmetic reduces money discrepancies, modular design supports rate revisions, and tests provide repeatable verification. Actual measurements and execution evidence must be added before final submission.